COFFA User Manual 1.0

Architecture of Reconfigurable Computing Lab

Fudan University

Content

1 Overview

COFFA is a comprehensive framework, as shown in Figure 1.

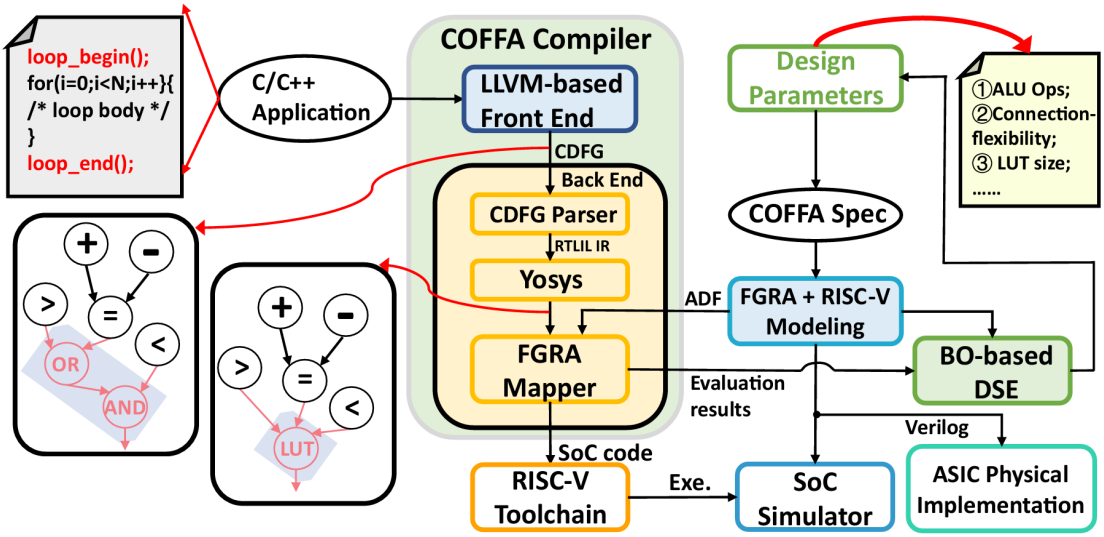

Figure 1 The COFFA framework.

1.1 Software Toolchain

The COFFA compiler takes the application C code, along with the manually annotated loop kernel intended to be executed on FGRA, as input. Then, the front-end tool converts the kernel into a Control Flow Graph (CFG), from which it extracts and converts control flow into data dependencies. Subsequently, the front-end tool identifies the fine-grained operations, performs coarse-grained optimization, and generates the fused-grained CDFG.

The back-end tool parses the generated CDFG, then uses Yosys to perform Boolean algebra optimization and LUT-based technology mapping on the identified fine-grained operations. The FGRA mapper maps the optimized CDFG to the ADF and generates the FGRA calling function, which contains the complete configuration data and FGRA programming interfaces for the CPU core. The calling function can then replace the original annotated loop kernel, and the transformed application code is compiled into an executable file by the RISC-V toolchain. Subsequently, the CPU can invoke the FGRA for the loop acceleration.

1.2 Hardware Modeling

As shown in Figure 2, the COFFA SoC consists of a RISC-V Rocket tile, an FGRA accelerator, and various control modules. Meanwhile, the multi-bank scratchpad memories (SPMs) are external to the FGRA to cache configuration and computation data. The load and store controller manages data transmission between the L2 cache and SPMs via TileLink, implemented by the DMA controller. In the case of multiple load instructions with the same virtual address but different physical addresses (i.e., loading data from the same memory into different SPMs), the DMA controller employs a data broadcast mechanism to transmit the data in a single DMA transmission to

multiple banks in the SPMs. The reservation station caches the instructions and dispatches them to each controller based on logical and software-annotated dependencies.

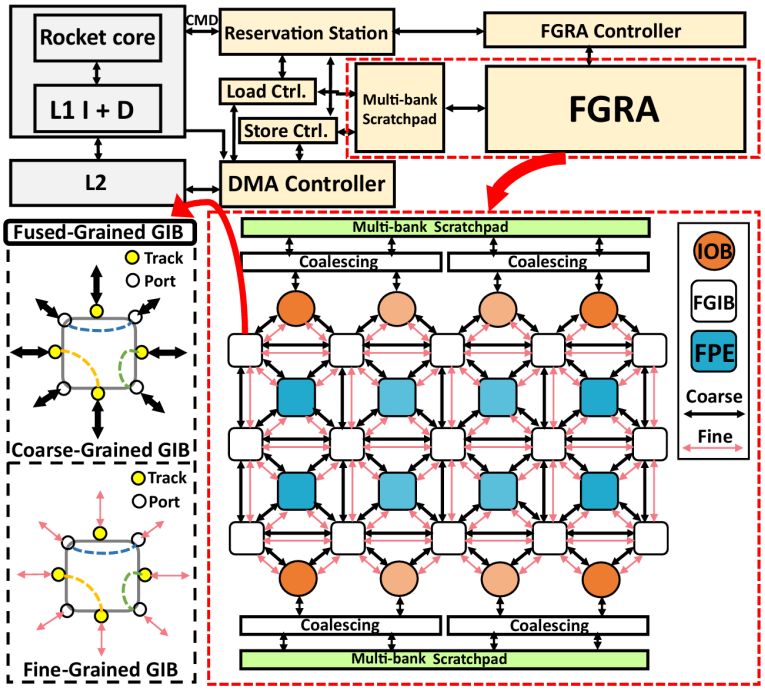

Figure 2 The COFFA SoC Architecture.

2 Installation

This chapter details the setup of the development environment. It covers the creation of a unified Conda environment, the compilation of all necessary tools like LLVM, and the setup of the Chipyard hardware simulator.

2.1 Unified Environment Setup with Conda

The first and most critical step is to create a dedicated Conda environment that houses all dependencies for the COFFA compiler.

2.1.1 Create the Configuration File

In your chosen working directory, create a new file named *environment.yml*. Copy and paste the following content into this file. It declaratively defines every tool and library required.

```yaml
# environment.yml: Defines all dependencies for the FGRA
toolchain
name: fgra-mapper-env

channels:
  - conda-forge

dependencies:
  # 1. Python Interpreter and Required Packages for the Mapper
  - python=3.12
  - pip
  - networkx
  - matplotlib

  # 2. C/C++ Compilers (for building LLVM and other projects)
  - gcc=15.1
  - gxx=15.1
  - binutils

  # 3. Build System Tools
  - cmake>=3.20
  - git
  - ninja

  # 4. C++ Libraries (for the COFFA compiler)
  - nlohmann_json=3.12.0
  - spdlog=1.15.3

  - pip:
    - pulp
```

Figure 3 Conda environment configuration file of COFFA

2.1.2 Create and Activate the Conda Environment

Open a terminal, navigate to the directory containing your *environment.yml* file, and run the following commands.

```
conda env create -f environment.yml -p conda

# Activate the environment for use
conda activate ./conda
```

Figure 4 Commands to create and activate the Conda environment

IMPORTANT! All subsequent commands in this manual must be executed within this activated Conda environment. Your terminal prompt should be prefixed with (fgra-mapper-env).

2.2 Core Components Installation

With the environment active, we will now compile and install the core components.

2.2.1 LLVM 10.0.0 Installation (from Source)

The App-Compiler (i.e., the front-end tool of COFFA compiler) requires a custom-built version of LLVM. Our Conda environment provides the necessary LLVM tools for this process.

```
# Step 1: Download and extract the LLVM source code
wget -O llvm-10.tar.gz https://github.com/LinHuan86/llvm-pro-
ject/archive/refs/tags/10.tar.gz
tar xvf llvm-10.tar.gz
mv llvm-project-10 llvm-10-src

# Step 2: Configure, compile, and install LLVM
mkdir llvm-10-build  # Create a separate build directory
cd llvm-10-src
cmake -DLLVM_ENABLE_PROJECTS='polly;clang' -G "Ninja" -S llvm
-B ../llvm-10-build
cd ../llvm-10-build
cmake --build .

# Install to a local directory.
# NOTE: Replace "/path/to/your/tools/llvm-10" with your actual
desired installation path.
make install DESTDIR=/path/to/your/tools/llvm-10
```

```
# Step 3: Update Environment PATH
# Add this line to your ~/.bashrc file, using the same path
from the `make install` command.
export PATH="/path/to/your/tools/llvm-10/usr/local/bin:$PATH"
```

Figure 5 Commands for compiling and installing the LLVM

Save the file and reload your shell configuration by running "*source ~/.bashrc*" or opening a new terminal.

2.2.2 Yosys Installation

COFFA relies on Yosys to perform Boolean algebra optimization. Hence, it is necessary to install Yosys before running the back-end tool of the COFFA compiler. As a popular synthesis tool, Yosys has a comprehensive document. Please refer to yosys/README.md at main · YosysHQ/yosys for a successful installation. After the installation, you can run the command *"Yosys"*, which will open the Yosys tool, as shown in Figure 6.

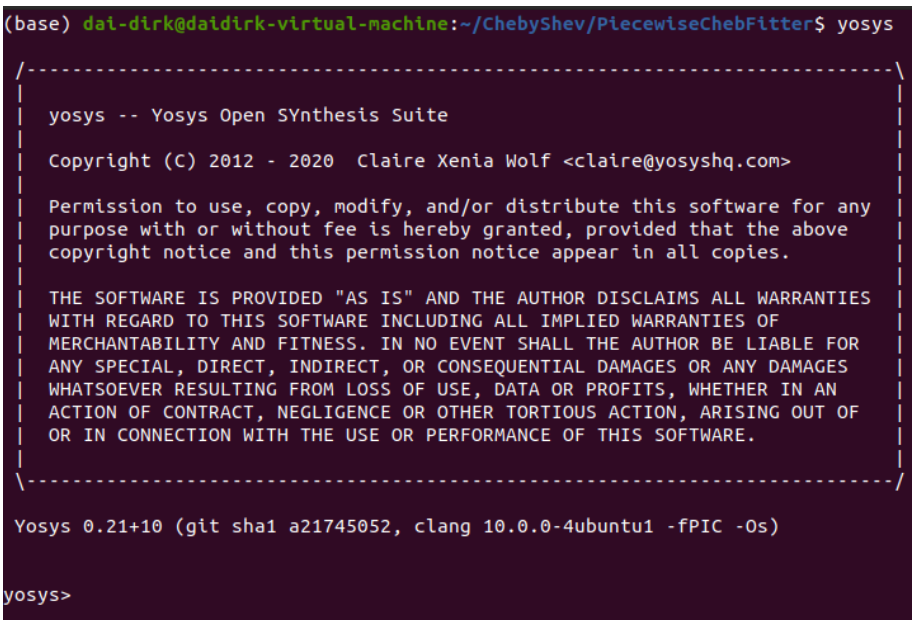

Figure 6 The example of running Yosys

2.2.3 COFFA Compiler Installation

Navigate to the directories for the app-compiler and fgra-compiler and run their respective build scripts.

```
# Build the App-Compiler
cd /path/to/your/app-compiler
./build.sh

# Build the FGRA-Mapper
cd /path/to/your/fgra-compiler
./build.sh
```

Figure 7 Commands for building the COFFA compilers

2.3 Chipyard Environment and Hardware Simulator Setup

The final installation step is to set up the Chipyard System-on-Chip (SoC) generator.

2.3.1 Clone and Initialize Chipyard

```
git clone https://github.com/ucb-bar/chipyard.git
cd chipyard
git checkout 1.10.0
./build-setup.sh riscv-tools
```

Figure 8 Commands for cloning and initializing the Chipyard repository

2.3.2 Integrate FGRA and Build the Simulator

```
# Add coffa in build.sbt
lazy val chipyard = (project in file("generators/chipyard"))
.dependsOn(testchipip, rocketchip, boom, hwacha,
sifive_blocks, sifive_cache, iocell, sha3, dsptools, rocket-
dsp-utils, gemmini, icenet, tracegen, cva6, nvdla, sodor,
ibex, fft_generator, constellation, mempress, barf, shuttle,
fgra)

lazy val fgra = (project in file("generators/fusion_mem/fgra-
mg"))
.dependsOn(rocketchip)
.settings(libraryDependencies ++= rocketLibDeps.value)
.settings(commonSettings)
```

Figure 9 Updating *build.sbt*

After installing the Chipyard, the first step is to add the COFFA project to the Chipyard dependency. This can be achieved by modifying the *build.sbt* file within the Chipyard (in the directory: ./chipyard/build.sbt). Subsequently, run the following commands to complete the simulator building.

```
# Navigate to the FGRA generator subdirectory within Chipyard
cd chipyard/generators/fgra

# Run the path setup script
./scripts/setup-paths.sh

# Source the main Chipyard environment script to set up paths
source ../../env.sh

# Build the Verilog hardware description from Chisel
./scripts/build-verilog.sh

# Build the Verilator C++ simulator from the generated Verilog
./scripts/build-verilator.sh
```

Figure 10 Commands for building the FGRA hardware simulator

3 End-to-End Simulation Flow

This chapter provides a complete, step-by-step walkthrough of the entire simulation flow, using the *adi* benchmark as a concrete example.

3.1 Workflow Overview

The process involves four main stages:

1. App-Compiler Stage: A C kernel is compiled into a Data Flow Graph (DFG).
2. FGRA-Compiler Stage: The DFG is mapped to the CGRA architecture, producing a hardware bitstream and control code.
3. SoC Application Stage: The generated code is integrated into a bare-metal C test harness.
4. Simulation Stage: The final application is compiled with the RISC-V toolchain and run on the Verilator-based simulator within Chipyard.

3.2 From C kernel to the Data Flow Graph

3.2.1 Input Kernel

The source C code contains the kernel to be accelerated. It uses *loop_begin()/loop_end()* pragmas to mark the target region and #pragma unroll to expose parallelism.

```
#define NUM 32
int A[NUM][NUM];
int B[NUM][NUM];
int X[NUM][NUM];
 //kernel 18
 void adi() {
  #ifdef CGRA_COMPILER
     loop_begin();
  #endif
     for (int i1=0 ; i1 < NUM ; i1++) {
        #pragma unroll 2
        for (int i2=0 ; i2 < NUM-2 ; i2++){
     X[i1][i2] = X[i1][i2] - X[i1][i2+1] * A[i1][i2] *
B[i1][i2+1];
     B[i1][i2] = B[i1][i2] - A[i1][i2] * A[i1][i2] *
B[i1][i2+1];
  }
  }
  #ifdef CGRA_COMPILER
     loop_end();
  #endif
 }
```

Figure 11 The *adi* C Kernel for Hardware Acceleration

3.2.2 Run the Compilation Script

The compile.sh script automates the process of generating LLVM IR and then running a custom LLVM pass (libCDFGPass.so) to extract the DFG.

```
export LLVM_HOME=/path/to/your/tools/llvm-10/usr/local/bin
export PATH=$LLVM_HOME:$PATH

clang -D CGRA_COMPILER -target i386-unknown-linux-gnu -c -
emit-llvm -O1 -fno-tree-vectorize -fno-unroll-loops $1.c -S -o
$1.ll

opt  -mem2reg -memdep -memcpyopt -lcssa -loop-simplify -licm -
loop-deletion -indvars -loop-simplifycfg -simplifycfg -
mergereturn -indvars $1.ll -S -o $1_gvn.ll

opt -load /home/linhuan/project/fgra/app-compiler/build/llvm-
pass/libCDFGPass.so -fn $2 -cdfg $1_gvn.ll -S -o $1_cdfg.ll
```

Figure 12 The *compile.sh* script for DFG generation

```
# Navigate to your adi benchmark directory
cd /path/to/benchmarks/test/adi

# Execute the script: `compile.sh <c_file_name>
<function_name>`
bash ../compile.sh adi adi
```

Figure 13 Core command for the front-end tool of the COFFA compiler

3.2.3 Outputs

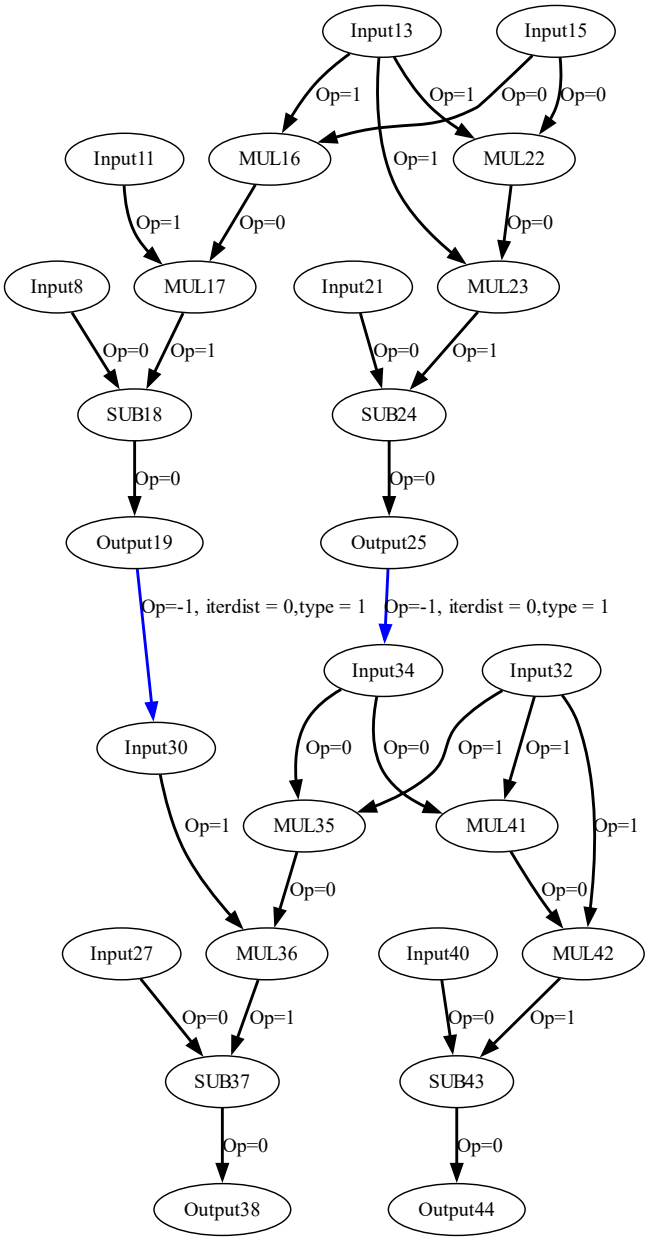

Figure 14 The generated DFG of the adi application

This stage generates an affine.dot (the DFG used by the next stage), which needs to be converted to JSON format using the *dot2json.sh* script. This JSON format DFG is used by the FGRA mapper.

3.3 From DFG to Hardware Configuration

3.3.1 Run the Mapper:

The *run.sh* script executes the FGRA Mapper, which maps the DFG onto the hardware. You must edit this script to point the -d flag to the affine.json file generated in the previous step.

```
# Core command within run.sh
./build/fgra-compiler ... -d
"/path/to/benchmarks/test/adi/affine.json" # <-- EDIT THIS
PATH
```

Figure 15 Location of DFG File, whose content needs to be changed for different examples

3.3.2 Outputs:

This stage generates two critical files for the final application:

```
void cgra_execute(void** din_addr, void** dout_addr)
    static unsigned short cin[122][3] __attribute__((aligned(16))) = {
        {0x2001, 0x0780, 0x9e12},
        {0x0040, 0x0010, 0x9e52},
        {0x0000, 0x0b30, 0x9e92},
        {0x8410, 0x0000, 0x9ed2},
    };

    load_cfg((void*)cin, 0x8000, 732, 0, 0);
    load_data(din_addr[0], 0x2000, 4088, 1, 0, 0, 1, 0);
    load_data(din_addr[0], 0x3000, 4088, 0, 0, 0, 1, 0);
    load_data(din_addr[1], 0x4000, 4092, 1, 0, 0, 1, 0);
    load_data(din_addr[1], 0x5000, 4092, 0, 0, 0, 1, 0);
    load_data(din_addr[2], 0x0, 4092, 1, 0, 0, 1, 0);
    load_data(din_addr[2], 0x1000, 4092, 0, 0, 0, 1, 0);
    config(0x0, 122, 0, 0);
    execute(0xff, 0, 0);
    store(dout_addr[0], 0x0, 4092, 1, 0, 0, 1, 0);
    store(dout_addr[0], 0x1000, 4092, 0, 0, 0, 1, 0);
    store(dout_addr[1], 0x4000, 4092, 1, 0, 0, 1, 0);
    store(dout_addr[1], 0x5000, 4092, 0, 0, 0, 1, 0);
}
```

Figure 16 The FGRA calling function (i.e., cgra_execute.c)

```
void* cgra_din_addr[3] = {A, B, X};
void* cgra_dout_addr[2] = {X, B};
cgra_execute(cgra_din_addr, cgra_dout_addr);
```

Figure 17 The function call of the CGRA execution flow

- cgra_execute.c: Containing the low-level hardware bitstream (the cin[...] array) and the API functions (load_cfg, execute, etc.) to control the FGRA.

- cgra_call.txt: Containing the high-level C code snippet for calling the FGRA with the correct data pointers (input and output data array), which indicating the required data and generated results.

3.4 Building the Final SoC Application

3.4.1 Prepare the Test Harness:

Use the provided file (in software/bareMetalC) as a template. This file contains the main function, data initialization, a software reference implementation for verification, and placeholders for the CGRA-specific code.

3.4.2 Integrate Generated Code (Critical Step):

```c
#include <string.h>
#include "include/ISA.h"

__attribute__((noinline))
void cgra_execute(void** din_addr, void** dout_addr)
{
    static unsigned short cin[47][3] __attribute__((aligned(16))) = {...

    load_cfg((void*)cin, 0xc000, 282, 0, 0);
    load_data(din_addr[0], 0x8000, 4092, 1, 0, 0, 1, 10);
    load_data(din_addr[0], 0x9000, 4092, 0, 0, 0, 1, 10);
    load_data(din_addr[1], 0xa000, 4096, 1, 0, 0, 1, 0);
    load_data(din_addr[1], 0xb000, 4096, 0, 0, 0, 1, 0);
    load_data(din_addr[2], 0x4000, 4096, 1, 0, 0, 1, 0);
    load_data(din_addr[2], 0x5000, 4096, 0, 0, 0, 1, 0);
    config(0x0, 47, 0, 0);
    execute(0xfb0, 0, 0);
    store(dout_addr[0], 0x4000, 4096, 1, 0, 0, 1, 0);
    store(dout_addr[0], 0x5000, 4096, 0, 0, 0, 1, 0);
    store(dout_addr[1], 0xa000, 4096, 1, 0, 0, 1, 0);
    store(dout_addr[1], 0xb000, 4096, 0, 0, 0, 1, 0);
}
// #define N 8
#define NUM 32
  int A[NUM][NUM]__attribute__((aligned(16)));
  int B[NUM][NUM]__attribute__((aligned(16)));
  int X[NUM][NUM]__attribute__((aligned(16)));
  int B_i[NUM][NUM]__attribute__((aligned(16)));
  int X_i[NUM][NUM]__attribute__((aligned(16)));
   //kernel 23
void adi() {
    for (int i1=0 ; i1 < NUM ; i1++) {
        for (int i2=0 ; i2 < NUM-1 ; i2++){
            X_i[i1][i2] = X_i[i1][i2] - X_i[i1][i2+1] * A[i1][i2] * B_i[i1][i2+1];
            B_i[i1][i2] = B_i[i1][i2] - A[i1][i2] * A[i1][i2] * B_i[i1][i2+1];
        }
    }
}
```

Figure 18 Example of copying the CGRA calling function to the target application

1. Integrate Bitstream: Open the generated cgra_execute.c. Copy the entire cgra_execute() function. Paste and replace the placeholder cgra_execute() function in adi.c(in software/bareMetalC).

```c
int main(){

//   int i,j;
  long long unsigned start;
  long long unsigned end;

    for (int i=0 ; i < NUM ; i++) {
        for (int j=0 ; j < NUM ; j++){
            A[i][j] = i + j;
            B[i][j] = i + j;
            X[i][j] = i + j;
            B_i[i][j] = i + j;
            X_i[i][j] = i + j;
        }
    }
    printf("Initialization finished!\n");

    //   printf("CPU add finished!\n");
    start = rdcycle();
    /* Run kernel. */
    adi();
    end = rdcycle();
    printf("It takes %d cycles for CPU to finish the task.\n", end - start);

    start = rdcycle();
    void* cgra_din_addr[3] = {A, B, X};
    void* cgra_dout_addr[2] = {X, B};
    cgra_execute(cgra_din_addr, cgra_dout_addr);
    volatile int result = fence(1);
    end = rdcycle();
    printf("It takes %d cycles for CGRA to finish the task(%d).\n", end - start, 1);
    printf("CGRA comput finished!\n");
    result = fence(1);

    printf("Checking results!\n");
    result_check();
    printf("Done!\n");

    return 0;

}
```

Figure 19 Example of copying the function call to the *main()* function

2. Integrate Function Call: Open the generated cgra_call.txt. Copy its contents. Paste them into the main() function of adi.c where the CGRA is called, replacing the placeholder call.

After these two modifications, adi.c becomes a complete, self-contained application ready for compilation.

3.5 Compiling and Running the Final Simulation

3.5.1 Compile the SoC Application:

Use the RISC-V toolchain provided by Chipyard to compile the final C file into an ELF binary.

```
# Navigate to the bare-metal software directory
cd /path/to/chipyard/generators/fgra/software/tests

# IMPORTANT! Ensure your application (e.g., adi-baremetal) is
listed in the Makefile. If not, add it.
./build.sh
```

Figure 20 Command to compile the final SoC application

3.5.2 Run the Application on the Verilator Simulator:

Execute the compiled binary on the hardware simulator you built in Chapter 1.

```
# Navigate to the Verilator simulation directory
cd /path/to/chipyard/sims/verilator

# Run the make command with the correct CONFIG and path to
your BINARY
make run-binary-debug CONFIG=FusionMemSoCConfig
BINARY=../../generators/fgra/software/tests/build/bareMetalC/a
di-baremetal
```

Figure 21 Command to execute the simulation through Verilator

3.5.3 Analyze the Final Simulation Output:

A successful run will produce output in your terminal like the following, confirming both performance and correctness:

```
Initialization finished!
It takes xxxxxx cycles for CPU to finish the task.
It takes yyyyy cycles for CGRA to finish the task(1).
CGRA comput finished!
Checking results!
Done!
```

Figure 22 Example terminal output from a successful simulation run

This output demonstrates a significant speedup from using the FGRA and verifies that the hardware results match the software reference implementation perfectly.

4 COFFA DSE Flow

The COFFA architecture has numerous design parameters, resulting in a vast design space, which makes manual exploration challenging and time-consuming. Therefore, we propose a Bayesian optimization (BO)-based DSE process to efficiently search and optimize the design parameters, ultimately generating a highly efficient

architecture. In this section, we provide installation instructions for relevant Bayesian optimization packages and offer a script to demonstrate the DSE process.

4.1 BO package installation.

Optuna is an automatic hyperparameter optimization framework that supports various sampling algorithms. Among them, the Tree-structured Parzen Estimator (TPE) is well suited for efficient search in complex design spaces.

```
# Install Optuna via pip:
conda activate fgra-mapper-env
pip install optuna

# Quick import test in Python
python -c "import optuna; print('Optuna version:',
optuna.__version__)"
```

Figure 23 Command to install Optuna

If Optuna is installed correctly, here is what the expected outputs look like:

```
Optuna version: 3.6.1
```

4.2 DSE FLOW

The following code defines our design space in script ***paraDefine.py***:

```
def paramspace(trial: optuna.Trial) -> dict:
    fgra_num_row = trial.suggest_int('fgra_num_row', 6, 6)
    fgra_num_colum = trial.suggest_int('fgra_num_colum', 6, 6)

    iob_mode = trial.suggest_int('iob_mode', 3, 3)
    max_delay_cg_iob = trial.suggest_int('max_delay_cg_iob', 2, 8)
    max_delay_fg_iob =  trial.suggest_int('max_delay_fg_iob', 8, 32)

    fgra_gib_num_track_cg =  trial.suggest_int('fgra_gib_num_track_cg', 1, 2)
    num_itrack_per_ipin_cg = trial.suggest_int('num_itrack_per_ipin_cg', 2, 6)
    num_otrack_per_opin_cg = trial.suggest_int('num_otrack_per_opin_cg', 2, 6)
    num_ipin_per_opin_cg = trial.suggest_int('num_ipin_per_opin_cg', 2, 6)
    diag_iopin_connect_cg = trial.suggest_int('diag_iopin_connect_cg', 0, 1)

    fgra_gib_num_track_fg =  trial.suggest_int('fgra_gib_num_track_fg', 1, 3)
    num_itrack_per_ipin_fg = trial.suggest_int('num_itrack_per_ipin_fg', 2, 6)
    num_otrack_per_opin_fg = trial.suggest_int('num_otrack_per_opin_fg', 2, 6)
    num_ipin_per_opin_fg = trial.suggest_int('num_ipin_per_opin_fg', 2, 6)
    diag_iopin_connect_fg = trial.suggest_int('diag_iopin_connect_fg', 0, 1)

    max_delay_cg_gpe = trial.suggest_int('max_delay_cg_gpe', 2, 8)
    max_delay_fg_gpe =  trial.suggest_int('max_delay_fg_gpe', 16, 32)
    num_input_lut = trial.suggest_int('num_input_lut', 2, 4)
```

Figure 24 The design parameters of COFFA

These statements define both the variable name and the scope of exploration. Our optimization objective is a weighted combination of *area*, *II*, *mapping failure rate*, *DFG*

latency, and *PE utilization*. The area is estimated using a rapid area evaluation model, which can be obtained during the Chisel compilation stage without relying on time-consuming commercial evaluation tools.

Next, we can run script ***dse-tpe.py*** to explore the effects of different parameter configurations using the TPE algorithm based on Bayesian optimization. Our default iteration count is set to 20. The iteration process takes approximately twenty minutes, after which the DSE algorithm provides the optimal architecture parameters, as shown below:

```
===== Best Solution =====
  Value: 373.4326797385621
  Params: {'fgra_num_row': 6, 'fgra_num_colum': 6, 'iob_mode':
3, 'max_delay_cg_iob': 7, 'max_delay_fg_iob': 25,
'fgra_gib_num_track_cg': 2, 'num_itrack_per_ipin_cg': 4,
'num_otrack_per_opin_cg': 6, 'num_ipin_per_opin_cg': 3,
'diag_iopin_connect_cg': 1, 'fgra_gib_num_track_fg': 1,
'num_itrack_per_ipin_fg': 2, 'num_otrack_per_opin_fg': 4,
'num_ipin_per_opin_fg': 6, 'diag_iopin_connect_fg': 0,
'max_delay_cg_gpe': 5, 'max_delay_fg_gpe': 19,
'num_input_lut': 4}
```

Figure 25 The DSE result, which shows the optimized parameter settings

5 COFFA FPGA Prototype (To be updated)

6 Running COFFA with Docker (To be updated)